Database I/O Models Guide

1. Buffered POSIX I/O (Double Page Cache)

In this model, standard read() and write() system calls are used. When the DBMS writes a page, the OS stores it in the Kernel Page Cache. This creates 'Double Buffering' because both the DBMS and the OS cache the same data in RAM. It's easy to use but wastes memory and can lead to data loss if a crash occurs before the OS flushes to disk.

```
#include <fcntl.h>
#include <unistd.h>

class DiskManager {
public:
    DiskManager(const std::string& db_file) {
        // Standard open without O_DIRECT
        db_io_ = open(db_file.c_str(), O_RDWR | O_CREAT, 0666);
    }

    void WritePage(page_id_t page_id, const char* page_data) {
        off_t offset = page_id * PAGE_SIZE;
        // pwrite copies data to OS Kernel Cache. Returns immediately.
        // Data is NOT guaranteed to be on disk!
        pwrite(db_io_, page_data, PAGE_SIZE, offset);
    }

    void ReadPage(page_id_t page_id, char* page_data) {
        off_t offset = page_id * PAGE_SIZE;
        // pread reads from OS cache if available, else blocks to fetch from disk.
        pread(db_io_, page_data, PAGE_SIZE, offset);
    }
private:
    int db_io_;
};
```

2. Direct I/O (O_DIRECT + fsync)

Using O_DIRECT bypasses the OS Kernel Page Cache. The DBMS reads/writes directly to the storage device. However, this requires aligned memory buffers and makes I/O calls fully synchronous (blocking). A thread calling pwrite() or fsync() will be completely blocked until the hardware completes the operation.

```
#ifndef _GNU_SOURCE
#define _GNU_SOURCE
#endif
#include <fcntl.h>
#include <unistd.h>

class DiskManager {
public:
```

```
    DiskManager(const std::string& db_file) {
        // O_DIRECT forces direct hardware access, bypassing kernel cache
        db_io_ = open(db_file.c_str(), O_RDWR | O_CREAT | O_DIRECT, 0666);
    }

    void WritePage(page_id_t page_id, const char* page_data) {
        // IMPORTANT: page_data memory MUST be aligned to PAGE_SIZE (e.g., alignas(4096))
        off_t offset = page_id * PAGE_SIZE;
        pwrite(db_io_, page_data, PAGE_SIZE, offset);

        // Force immediate write to hardware. The calling thread BLOCKS here.
        fsync(db_io_);
    }
private:
    int db_io_;
};
```

3. Worker Thread Pool (Simulated Async I/O)

To prevent blocking the main DBMS threads (like query execution), we can spawn a pool of worker threads. Main threads submit I/O requests to a queue. Worker threads dequeue requests and execute the blocking pread/pwrite/fsync. While this achieves concurrency, it comes with the high overhead of OS thread context switching and lock contention.

```
#include <thread>
#include <vector>
#include <queue>
#include <mutex>
#include <condition_variable>
#include <functional>

class IOWorkerPool {
public:
    IOWorkerPool(size_t num_workers) {
        for(size_t i = 0; i < num_workers; ++i) {
            workers_.emplace_back([this]() {
                while(true) {
                    std::function<void()> task;
                    {
                        std::unique_lock<std::mutex> lock(queue_mutex_);
                        // Block worker thread until a task is available
                        condition_.wait(lock, [this]{ return stop_ || !tasks_.empty(); });
                        if(stop_ && tasks_.empty()) return;

                        task = std::move(tasks_.front());
                        tasks_.pop();
                    }
                    task(); // Execute blocking I/O (e.g., fsync or pread)
```

```
                }
            });
        }
    }

    void Submit(std::function<void()> task) {
        {
            std::unique_lock<std::mutex> lock(queue_mutex_);
            tasks_.push(std::move(task));
        }
        condition_.notify_one(); // Wake up one worker
    }

    // Usage in DBMS:
    // io_pool.Submit([this, page_id, data]() { disk_manager_->WritePage(page_id, data); });
};
```

4. io_uring (True Kernel Async I/O)

io_uring is a modern Linux kernel feature providing true asynchronous I/O. Instead of threads and locks, it uses two lockless ring buffers shared between user-space and the kernel: a Submission Queue (SQ) and a Completion Queue (CQ). The DBMS places I/O requests in the SQ and checks the CQ for completed requests. This eliminates context switches, thread overhead, and blocking entirely.\n\nNote: The code below uses 'liburing', the standard C wrapper library for io_uring, rather than calling the raw Linux kernel syscalls directly, as it abstracts away complex memory mapping and CPU barriers.

```
#include <liburing.h> // Requires liburing
#include <fcntl.h>
#include <unistd.h>
#include <iostream>

class IOUringManager {
public:
    IOUringManager(const std::string& db_file) {
        db_io_ = open(db_file.c_str(), O_RDWR | O_CREAT | O_DIRECT, 0666);
        // Initialize the lockless ring buffers with 32 entries
        io_uring_queue_init(32, &ring_, 0);
    }

    void AsyncWrite(page_id_t page_id, const char* page_data) {
        struct io_uring_sqe *sqe = io_uring_get_sqe(&ring_);
        off_t offset = page_id * PAGE_SIZE;

        // 1. Prepare write request in the Submission Queue (SQ)
        io_uring_prep_write(sqe, db_io_, page_data, PAGE_SIZE, offset);

        // 2. Set user data (page_id) to track the request later
        io_uring_sqe_set_data(sqe, (void*)(uintptr_t)page_id);

        // 3. Submit request to the kernel. This DOES NOT BLOCK.
        io_uring_submit(&ring_);
    }

    void PollCompletions() {
        struct io_uring_cqe *cqe;
        // Check Completion Queue (CQ) for finished hardware I/O
        while (io_uring_peek_cqe(&ring_, &cqe) == 0) {
            if (cqe->res < 0) {
                std::cerr << "Async I/O Error: " << -cqe->res << "\n";
            } else {
                // Recover the page_id we set earlier
                page_id_t completed_page = (page_id_t)(uintptr_t)io_uring_cqe_get_data(cqe);
                std::cout << "Write completed for page " << completed_page << "\n";
            }
```

```
            // Mark completion as processed
            io_uring_cqe_seen(&ring_, cqe);
        }
    }
private:
    int db_io_;
    struct io_uring ring_;
};
```

Database I/O Models Guide

Architectural Impact: Buffer Pool & Query Execution

Moving from synchronous I/O (Models 1 & 2) to asynchronous I/O (Models 3 & 4) forces a fundamental architectural change in how the BufferPoolManager and Query Executor are designed.

Synchronous I/O (Models 1 & 2)

In a synchronous architecture, the BufferPoolManager operates via simple, blocking function calls:\n1. The query thread requests a page: buffer_pool->FetchPage(5)\n2. The buffer pool directly calls disk_manager->ReadPage(5)\n3. The OS blocks (pauses) the query thread until the disk finishes fetching the data.\n4. When the OS wakes the thread up, the data is returned and the query continues.\n\nProblem: High concurrency requires thousands of threads, leading to excessive OS context-switching overhead.

Asynchronous I/O (Models 3 & 4)

In asynchronous models, calls to the DiskManager return instantly before the data is ready. This requires an Event-Driven Architecture with a user-space Task Scheduler:\n\n1. State Tracking: The buffer pool allocates a frame, sets its state to IO_IN_PROGRESS, and calls AsyncRead().\n2. Yielding (Scheduler): The query thread cannot block. Instead, it yields its execution (e.g., via C++ Coroutines: co_await FetchPage(5)). The thread is then free to execute other pending queries.\n3. Background Poller: A dedicated background Event Loop continuously polls the io_uring Completion Queue (CQ) or listens to worker threads.\n4. Callbacks / Wake Up: When the Event Loop detects that the disk read has finished, it updates the page state to READY, and signals the Scheduler to wake up the original query so it can resume execution.\n\nThis complexity is why asynchronous engines are harder to build, but they offer vastly superior throughput under heavy loads.

Comparison Summary

Model	Blocking?	Memory Overhead	Context Switches	Complexity
Buffered	Soft (Cache)	High (Double)	Low	Low
O_DIRECT	Yes (Hard)	Low	Low	Low-Med
Workers	No (Main)	Med (Stacks)	High (Locks)	Medium
io_uring	No (Never)	Low (Ring)	Zero	High (Async)